

《系统仿真与 matlab》综合试题

题 目: 战争模型

编 号: 30

姓 名 _____

班 级 _____

学 号 _____

联系方式 _____

得分项	创新性	工作量	代码可读性	报告写作	总分
分数					

目录

1. 引言.....	3
2. 模型建立.....	3
2.1. 问题介绍——战争模型.....	3
2.2. 建模思路.....	3
2.3. 模型概念与符号的定义.....	3
2.4. 模型建立.....	4
2.5. 在三种情况中应用模型.....	5
2.6. 胜负判断条件.....	5
3. 程序亮点展示.....	5
4. 仿真与程序设计思路.....	6
4.1. 软件基础与程序运行环境.....	6
4.2. 模型求解思路.....	6
4.2.1. 理论与实际模型的程序化构建.....	6
4.2.2. 求解器的选择.....	7
4.2.3. 求解代码展示.....	7
4.3. 程序设计思路.....	8
4.3.1. 参数输入的实现.....	9
4.3.2. 仿真运行的实现:	10
4.3.3. 人数-时间曲线图的绘制实现.....	11
4.3.4. 结果判断的实现.....	12
4.3.5. 人数模拟散点图的绘制.....	13
4.3.6. 人数变化柱状图的绘制.....	15
5. 程序运行指南.....	17
6. 程序运行实例分析.....	17
6.1. 正规战对正规战.....	17
6.2. 正规战对游击战.....	18
6.3. 游击战对游击战.....	19
6.4. 混合战.....	20
7. 总结与感悟.....	21

1. 引言

影响一场战争的因素是多方面的，如双方参与人数、士兵作战素质等。而具体到一次战争的胜负，部队采取的作战方式同样至关重要，此时作战空间同样成为讨论一个作战部队整体战斗力的一个不可忽略的因素。因此预测战争走向是一个较为复杂的难题。

本文中通过对抽象出的战争中的一些关键要素进行微分方程的数学建模，并且通过 matlab 对数学模型进行仿真求解，最后利用 matlab app designer 制作 app，将模型仿真求解的结果可视化，可以通过人数曲线以及人员模拟散点图直观的看到随着时间的推移参战人数的变化。

2. 模型建立

2.1. 问题介绍——战争模型

影响一个军队战斗力的因素是多方面的，比如士兵人数、单个士兵的作战素质以及部队的军事装备，而具体到一次战争的胜负，部队采取的作战方式同样至关重要，此时作战空间同样成为讨论一个作战部队整体战斗力的一个不可忽略的因素。通过建立战争模型，以预测一场战争的大致结局。

2.2. 建模思路

战场情况复杂多变，本文中将战斗分为正规战、游击战、混合战三种情况。不难发现，混合战包括了正规战和游击战。因此，本文将正规战、游击战统一为混合战，也就是把正规战和游击战抽象为混合战的一种特殊情况，增设了一个变量 m 表示整体人数中参与游击战的人数比例，当 $m=0$ 时，就是只有正规战的情况；当 $m=1$ 时，就是只有游击战的情况；而当 m 在 $0-1$ 之间时，就是混合战。这样用混合战一种模型，就可以有效地表示正规战、游击战、混合战三种情况。这种思路的数学模型简洁，代码复用率高。

2.3. 模型概念与符号的定义

概念与符号	定义
正规战(regular)	每一方士兵的活动均公开，处于对方士兵的监视与杀伤范围之内，一旦一方的某个士兵被杀伤，对方的火力立即转移到其他士兵身上。
游击战(guerilla)	每一方士兵的活动均具有隐蔽性，对方的射击行为局限在

	某个范围考虑可以被认为是盲目的。
混合战	正规战和游击战两种情形的混合。
$x(t), y(t)$	甲(x)、乙(y)两方正规军和游击队在 t 时刻的总兵力。
α, β	甲(α)、乙(β)两方的非战斗减员率。
$u(t), v(t)$	甲(u)、乙(v)两方的增援率,增援率是以时间为变量的函数。
r_x, r_y	甲(x)、乙(y)两方的射击率。
p_x, p_y	甲(x)、乙(y)两方的命中率。
S_x, S_y	甲(x)、乙(y)两方游击队的有效活动区域面积。
m_x, m_y	甲(x)、乙(y)两方的总军队中游击队人数的占比。

2.4. 模型建立

如上文所言，本文将正规战、游击战、混合战统一为混合战的情形。因此只对混合战进行建模。

考虑到增援率是有关时间的函数，本模型假设

$$u(t) = \frac{ux}{t^2}, v(t) = \frac{vy}{t^2}$$

即时刻 t 的增援人数与当前人数成正比，与当前时刻的平方成反比，这也符合随着战争的持续后备资源逐渐被消耗增援越来越少的情况。

为简化模型，本文假设一方所受到的攻击会按照正规军和游击队的人数比例分配到正规军和游击队，同时也假设游击队和正规军的命中率和射击率相等。因此，每个时间间隔 t 正规军的死亡人数为

$$(1 - m)y(t)r_y p_y, (1 - m)x(t)r_x p_x$$

游击队的攻击力与其活动范围成正比，与敌方活动范围成反比，因此每个时间间隔 t 游击队的死亡人数为

$$my(t)r_y p_y \frac{S_y}{S_x}, mx(t)r_x p_x \frac{S_x}{S_y}$$

因此，再考虑到非战斗减员率的影响，结合上述的死亡人数以及增援人数可以得到：

甲军在一个时间间隔内的人员变化率的微分方程建模为：

$$\frac{dx(t)}{dt} = -(1 - m)y(t)r_y p_y - my(t)r_y p_y \frac{S_y}{S_x} + u(t, x) - \alpha x(t)$$

同理，乙军在一个时间间隔内的人员变化率的微分方程建模为：

$$\frac{dy(t)}{dt} = -(1 - m)x(t)r_x p_x - mx(t)r_x p_x \frac{S_x}{S_y} + v(t, y) - \beta y(t)$$

整理可得：

$$\begin{aligned}\frac{dx(t)}{dt} &= -\alpha x(t) - ((1-m)r_y p_y + m r_y p_y \frac{S_y}{S_x}) y(t) + \frac{ux(t)}{t^2} \\ \frac{dy(t)}{dt} &= -\beta y(t) - ((1-m)r_x p_x + m r_x p_x \frac{S_x}{S_y}) x(t) + \frac{vy(t)}{t^2}\end{aligned}$$

2.5. 在三种情况中应用模型

本章讨论了如何将统一的混合战模型应用于正规战、游击战等情况的方法。在某些设置中，在公式分母里添加了一个极小值防止出现分母为零的情况。

正规战：设置双方的游击部队人数为 0。

游击战：设置双方的正规部队人数为 0。

正规战对游击战：设置一方游击部队人数为 0，另一方正规部队人数为 0。

混合战：依据需求任意设置数值。

2.6. 胜负判断条件

本文中估计胜利条件为战争结束时，兵力大于敌方两倍的军队获胜。

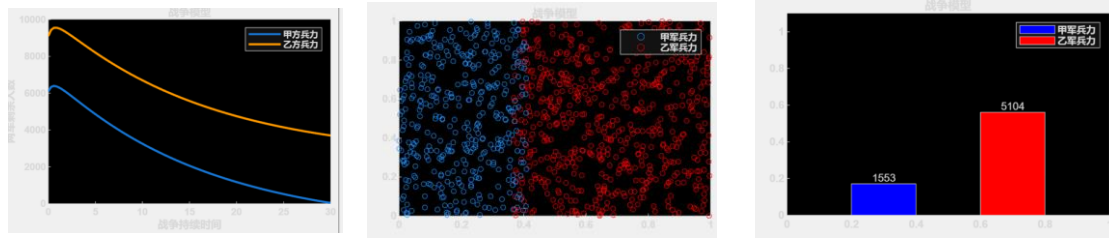
若没有一方满足胜利条件，则判定为两败俱伤。

3. 程序亮点展示

1. 本程序的模型建立将正规战和游击战抽象为混合战的特殊情况，在保证模型仿真符合实际情况的基础上大大简化了模型复杂度。
2. 本程序在模型求解时考虑了战争人数较大带来的微分方程刚度问题，采用了刚性求解器 `ode15s` 求解。
3. 本程序的界面清晰美观，功能明确具体，用户可以轻易上手操作。



4. 本程序使用多种方式进行输入输出,输入可以自定义也可以使用随机生成数据或内部预设数据,输出可以通过曲线图、散点图和柱状图观察到。



5. 本程序进行了很多动画效果的制作。输出结果的三种呈现形式都制作了对应的动画效果,更能直观的看到随时间推移人数的变化。同时胜负判决结果的展示也使用了动效表现,更加引人注目。

4. 仿真与程序设计思路

4.1. 软件基础与程序运行环境

软件在 win11 系统中使用 matlabR2024a 编写

图形化可视界面的搭建依赖于 matlab app designer。

4.2. 模型求解思路

4.2.1. 理论与实际模型的程序化构建

由前文的模型建立可知,建立出的模型微分方程为:

$$\begin{aligned}\frac{dx(t)}{dt} &= -\alpha x(t) - ((1-m)r_y p_y + m r_y p_y \frac{S_y}{S_x}) y(t) + \frac{ux(t)}{t^2} \\ \frac{dy(t)}{dt} &= -\beta y(t) - ((1-m)r_x p_x + m r_x p_x \frac{S_x}{S_y}) x(t) + \frac{vy(t)}{t^2}\end{aligned}$$

将此式转为矩阵形式可得：

$$\begin{bmatrix} \frac{dx(t)}{dt} \\ \frac{dy(t)}{dt} \end{bmatrix} = \begin{bmatrix} -\alpha & -r_y p_y \left((1-m) + m \frac{S_y}{S_x} \right) \\ -r_x p_x \left((1-m) + m \frac{S_x}{S_y} \right) & -\beta \end{bmatrix} \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} + \begin{bmatrix} \frac{ux(t)}{t^2} \\ \frac{vy(t)}{t^2} \end{bmatrix}$$

$$\text{令 } A = \begin{bmatrix} -\alpha & -r_y p_y \left((1-m) + m \frac{S_y}{S_x} \right) \\ -r_x p_x \left((1-m) + m \frac{S_x}{S_y} \right) & -\beta \end{bmatrix},$$

$$B(t, x, y) = \begin{bmatrix} \frac{ux(t)}{t^2} \\ \frac{vy(t)}{t^2} \end{bmatrix}$$

则有

$$\begin{bmatrix} \frac{dx(t)}{dt} \\ \frac{dy(t)}{dt} \end{bmatrix} = A \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} + B$$

理论模型如上，在实际求解时还考虑了以下几点：

1. 考虑到人数必须为正整数，当 x 或 y 的值小于等于 0 时，在计算 A 的值的程序中，加入条件判断：若 $x \leq 0$ ，则令 A 的第一行为 0，使其导数为 0 不再减小；同理，若 $y \leq 0$ ，令 A 的第二行为 0。
2. 在实际仿真过程中出现了死亡人数过快的的问题，为贴合实际，在战斗减员率的计算式前增加了一个系数 k ，本模型中定 $k=1/4$ 。
3. 在双方都只进行正规战时，双方的活动面积 S 为无关变量，但是若 $S=0$ ，在程序中考考虑到分母不能为 0，因此在 A 中的两个分母 S 项都加了 0.01 以防止程序求解发生数学错误。

4.2.2. 求解器的选择

在求解战争模型微分方程时，若军队数量较多，可能会出现微分方程的刚度问题，即微分方程中有的部分变化很快，有的部分变化很慢。这会导致传统的数值方法（如显式欧拉法）需要非常小的步长才能保持数值稳定性。

此时，对于传统微分方程较为友好的 `ode45` 求解器对刚性问题的求解并不适合，因此我选用了适合求解刚性微分方程的 `ode15s` 作为求解器，可以正常处理人数较多的战争模型仿真。

4.2.3. 求解代码展示

```
1. function [t,y]=probsolve(Adata, Bdata, time)
2.     initial = [Adata.initial_num; Bdata.initial_num];
```

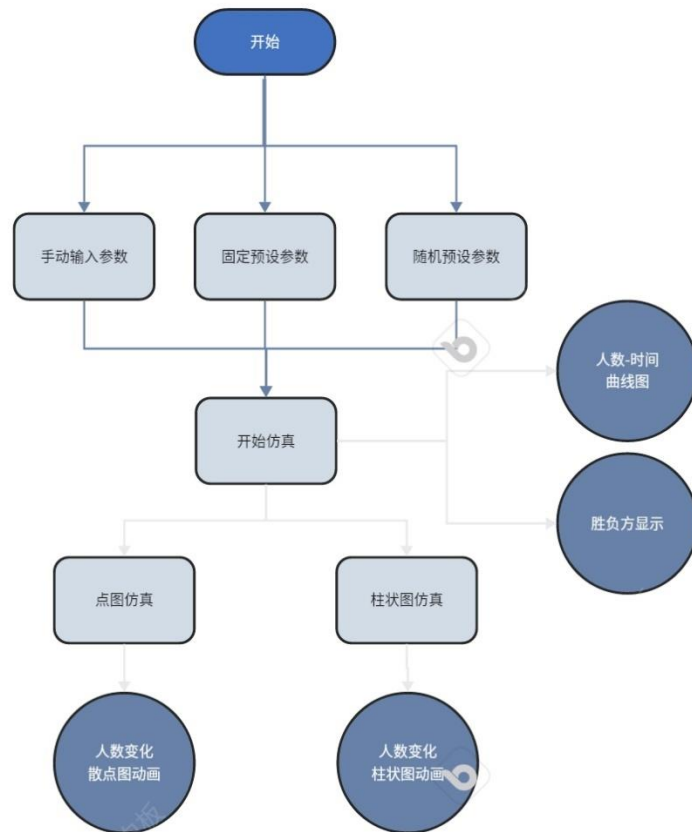
```

3.         options = odeset('RelTol',1e-6,'AbsTol',1e-8);
4.         [t,y]=ode15s(@(t, y) stamtrx(t, y, Adata, Bdata)*[y(1);y(2)]+.
..
5.             Bmtrx(t, y, Adata, Bdata),0:time/100:time,initial,options)
;
6.     end
7.
8.     %用于求状态空间矩阵 A 的函数
9.     function mtrx = stamtrx(~, y, Adata, Bdata)
10.        %计算状态空间矩阵
11.        k=0.1;
12.        mtrx=[-Adata.reduce_rate,-k*((1-Adata.gue_rate)...
13.            *Bdata.hit_rate*Bdata.shoot_rate+...
14.            Adata.gue_rate*Bdata.hit_rate*Bdata.shoot_rate*...
15.            Bdata.area/(Adata.area+0.01));
16.
17.            -k*((1-
Bdata.gue_rate)*Adata.hit_rate*Adata.shoot_rate+...
18.            Bdata.gue_rate*Adata.hit_rate*Adata.shoot_rate*...
19.            Adata.area/(Bdata.area+0.01)),-Bdata.reduce_rate];
20.        %如果人数等于 0 那么让导数也等于 0
21.        if y(1)<=0
22.            mtrx(1, :) = 0;
23.        end
24.        if y(2)<=0
25.            mtrx(2, :) = 0;
26.        end
27.    end
28.
29.    %用于求状态空间矩阵 B 的函数
30.    function Bmtrx=Bmtrx(t, y, Adata, Bdata)
31.        Bmtrx=[Adata.add_rate*y(1)/(t+1)^3;...
32.            Bdata.add_rate*y(2)/(t+1)^3];
33.    end

```

4.3. 程序设计思路

本程序的主要功能有：接收手动输入或者随机生成或者程序预设的参数，利用参数进行微分方程的求解，使用求解得到的结果绘制时间-人数曲线图。此外，为更加清晰明确的反映两军的人数变化与比较，我还制作了人员模拟散点图和人数变化柱状图。为了使图的结果更加美观，在三种结果展示时我都增加了动画效果。程序的主体内容如下图。下面将详细介绍每部分的程序实现思路。



4.3.1. 参数输入的实现

参数输入有三种形式：手动输入、随机生成和程序预设，其中手动输入是直接由 matlab app designer 实现的，无需编程。随机生成和程序预设的思路都是将整个 app 的句柄作为参数传入函数中，在函数中改变各输入框的值。

随机生成使用 `unifrnd` 函数在确定的范围生成一个值，再使用 `round` 函数确保其小数位数。随机生成参数的代码如下，固定预设思路一致，实现更简单，这里就不赘述了。

```

1. function set_rand_data(app)
2.     %设置 A 的随机参数
3.     app.A_add_rate.Value = round(unifrnd(0.001, 0.5),3); % 增援率
4.     app.A_hit_rate.Value = round(unifrnd(0.4, 0.9),2); % 命中率
5.     app.A_shoot_rate.Value = round(unifrnd(0.1, 0.9),2); % 射击率
6.     app.A_initial_num.Value = round(unifrnd(100, 10000)); % 初始兵力
7.     app.A_reduce_rate.Value = round(unifrnd(0.001, 0.05),3); % 非战斗减员率
8.     app.A_gue_rate.Value = round(unifrnd(0, 1),2);%游击战比率
9.     app.A_area.Value = round(unifrnd(10, 50));%面积
10.    %设置 B 的随机参数
  
```

```

11.      app.B_add_rate.Value = round(unifrnd(0.001, 0.5),3); % 增援率
12.      app.B_hit_rate.Value = round(unifrnd(0.4, 0.9),2); % 命中率
13.      app.B_shoot_rate.Value = round(unifrnd(0.1, 0.9),2); % 射击率
14.      app.B_initial_num.Value = round(unifrnd(100, 10000)); % 初始兵力
15.      app.B_reduce_rate.Value = round(unifrnd(0.001, 0.05),3); % 非战斗减员率
16.      app.B_gue_rate.Value =round(unifrnd(0, 1),2);%游击战比率
17.      app.B_area.Value =round(unifrnd(10, 50));%面积
18.      %仿真时间
19.      app.wartime.Value = round(unifrnd(10,100));
20.      end

```

4.3.2. 仿真运行的实现

首先将各输入框中的参数读取，之后调用上文分析过的求解程序代码中进行微分方程求解得到 $[t,y]$ 的值，程序中将读入的战斗时间均分为 100 份，其中 t 为当前的时间点， y 为一个二维数组，记录 t 时刻对应的甲乙军剩余人数。之后，将 t,y 和 app 句柄作为参数传入曲线绘制函数中绘制人数-时间曲线，最后进行胜负判断并显示胜负结果。

代码实现如下：

```

1.      function [t,y]=calculate(app)
2.          %初始化灯的颜色
3.          app.A_Lamp.Color=[1,0,0];
4.          app.B_Lamp.Color=[1,0,0];
5.          %收集 A 的参数
6.          Adata.add_rate=app.A_add_rate.Value ; % 增援率
7.          Adata.hit_rate=app.A_hit_rate.Value ; % 命中率
8.          Adata.shoot_rate=app.A_shoot_rate.Value ; % 射击率
9.          Adata.initial_num=app.A_initial_num.Value ; % 初始兵力
10.         Adata.reduce_rate=app.A_reduce_rate.Value ; % 非战斗减员率
11.         Adata.gue_rate=app.A_gue_rate.Value ;%游击战比率
12.         Adata.area=app.A_area.Value ;%面积
13.         %收集 B 的参数
14.         Bdata.add_rate=app.B_add_rate.Value ; % 增援率
15.         Bdata.hit_rate=app.B_hit_rate.Value ; % 命中率
16.         Bdata.shoot_rate=app.B_shoot_rate.Value ; % 射击率
17.         Bdata.initial_num=app.B_initial_num.Value ; % 初始兵力
18.         Bdata.reduce_rate=app.B_reduce_rate.Value ; % 非战斗减员率
19.         Bdata.gue_rate=app.B_gue_rate.Value ;%游击战比率
20.         Bdata.area=app.B_area.Value ;%面积
21.         %仿真时间

```

```

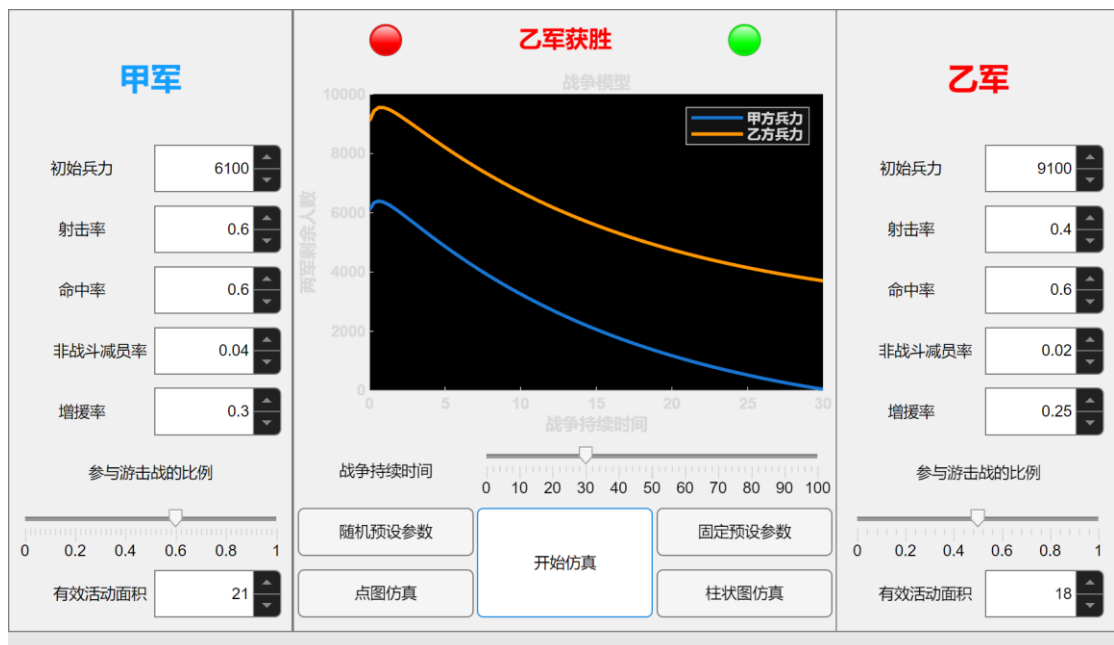
22.     wartime=app.wartime.Value;
23.
24.     [t, y] = probsolve(Adata, Bdata, wartime);
25.     drawline(app,t, y);
26.     winner_decd(app,y(end, 1),y(end, 2));
27. end

```

4.3.3. 人数-时间曲线图的绘制实现

首先需要清空坐标轴，设置线条样式和图例，接着根据甲乙军初始的人数的最大值和战斗时设置为坐标轴范围，保证曲线图在图形视窗中的可见性。

在线条绘制时，我使用了 `pause` 函数制作延时效果实现了图像的逐渐绘制的动效，在 `for` 循环中每一次都绘制一个节点到下一个节点的图像，然后延时一小段时间，以此实现曲线动效。动画效果可见演示视频.mp4



代码实现如下：

```

1.     function drawline(app,t, y)
2.         cla(app.UIAxes);
3.         %设置线条
4.         line1 = animatedline(app.UIAxes, 'Color', '#1976D2', 'LineWidth
h', 2);
5.         line2 = animatedline(app.UIAxes, 'Color', '#FF9800', 'LineWidth
h', 2);
6.         %设置坐标轴范围
7.         axis(app.UIAxes, [0, t(end), 0 ...
8.             , 1.1 * max(y(1, 1) , y(1, 2))]);

```

```

9.      %设置图例
10.     legend(app.UIAxes, "甲方兵力", "乙方兵力");
11.     len_t = length(t);
12.     for k = 1:len_t
13.         addpoints(line1, t(k), y(k, 1));
14.         addpoints(line2, t(k), y(k, 2));
15.         drawnow limitrate;
16.         pause(1.5 / len_t);
17.     end
18. end

```

4.3.4. 结果判断的实现

我们将结果的判定定义为：如果在战争结束时有一方的剩余人数超过了另一方的 2 倍，则判定该方为胜方，如果两方都不满足胜方的条件，则判定为两败俱伤。

结果的可视化主要通过图像绘制界面上方的指示灯和文字来显示，其中红灯表示失败、绿灯表示胜利、黄灯表示两败俱伤。效果如图。



同时，在结果显示时，我还增加了胜方的灯的闪烁动效使可视化效果更明显。动画效果可见演示视频.mp4

代码如下：

```

1.  function winner_decd(app,A_num, B_num)
2.      if A_num>=2*B_num
3.          app.B_Lamp.Color=[1,0,0];
4.          app.winner_show.Text="甲军获胜";
5.          app.winner_show.FontColor=[0,0,1];
6.          for i = 1:3
7.              app.A_Lamp.Color=[1,0,0];%red
8.              pause(0.2);
9.              app.A_Lamp.Color=[0,1,0];%green
10.             pause(0.2);
11.         end
12.
13.     elseif B_num>=2*A_num
14.         app.A_Lamp.Color=[1,0,0];
15.         app.winner_show.Text="乙军获胜";
16.         app.winner_show.FontColor=[1,0,0];
17.         for i = 1:3
18.             app.B_Lamp.Color=[1,0,0];%red
19.             pause(0.2);

```

```

20.         app.B_Lamp.Color=[0,1,0];%green
21.         pause(0.2);
22.     end
23.     else
24.         app.winner_show.Text="两败俱伤";
25.         app.winner_show.FontColor=[1,0.84,0];
26.         for i = 1:3
27.             app.B_Lamp.Color=[0,0,0];
28.             app.A_Lamp.Color=[0,0,0];
29.             pause(0.2);
30.             app.B_Lamp.Color=[1,0.84,0];
31.             app.A_Lamp.Color=[1,0.84,0];
32.             pause(0.2);
33.         end
34.     end
35. end

```

4.3.5. 人数模拟散点图的绘制

除了常规的人数时间图像，我还制作了用散点表示人数，用散点覆盖面积对比表示双方人数之比的散点图来更直观的感受双方力量的变化过程。

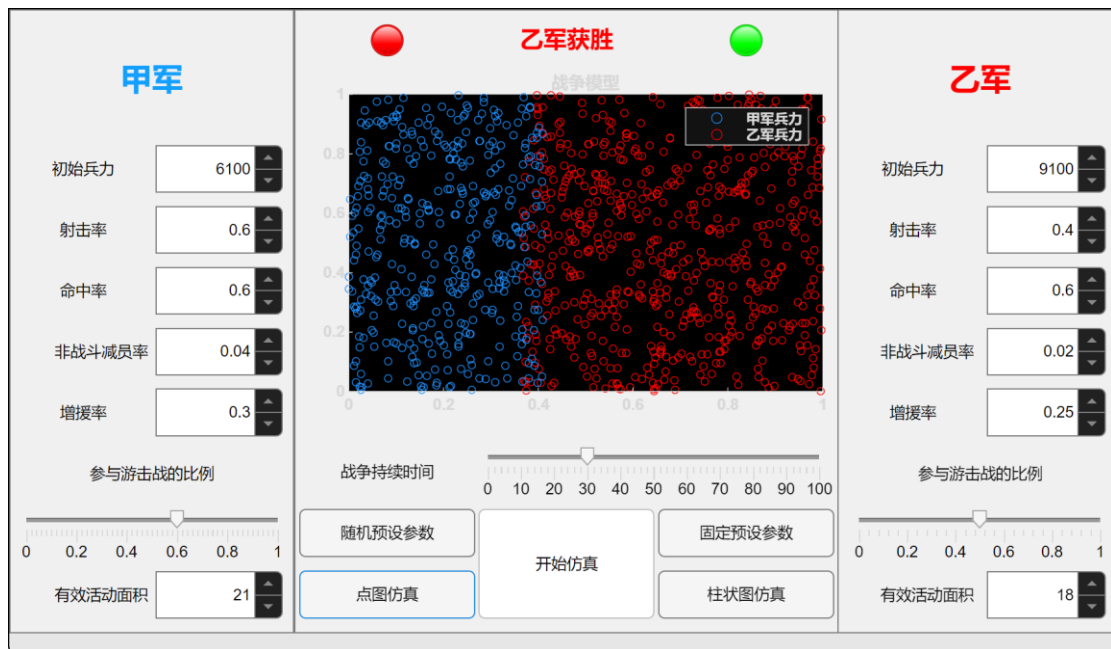
在这个地方，由于之前的计算都是在微分方程求解的函数中进行，得到的[t,y]矩阵也是函数内的变量，因此我在 APP 中新增设了全局属性 t,y 用来记录当前参数仿真后返回的 t 和 y，并将其传入绘制散点图的函数中用于绘制。

实现的思路大致如下：

首先设置好需要显示的帧数 k，这里设置为 50 帧。之后进入循环进行每帧的图像绘制。

首先从传入的全局属性中计算这一帧时的两军人数，并计算两军人数的比例。为了防止人数过多导致图中散点太多影响可视化效果，因此我将图上散点数量固定设置为 1000 个，通过不同颜色散点的比例来表示人员力量的对比。

之后根据人员比例确定在坐标轴上的散点区域，并在此区域按比例的数量生成随机的 XY 坐标并进行点的绘制，每一帧绘制后延时一小段时间。动画效果可见演示视频 .mp4。



代码实现如下：

```

1. function draw_point(app,~,y)
2.     % 添加坐标轴标签
3.     xlabel(app.UIAxes, ' ');
4.     ylabel(app.UIAxes, ' ');
5.     %设置显示的帧数
6.     K=50;
7.     for i = 1:K
8.         %清空并设置坐标轴范围
9.         cla(app.UIAxes);
10.        axis(app.UIAxes, [0,1,0,1]);
11.        %计算此时甲乙两军的数量
12.        Anum=y(i*round((101/K)),1);
13.        Bnum=y(i*round((101/K)),2);
14.        xmax=round(Anum/(Anum+Bnum),3);
15.        %生成甲军的散点
16.        xa_range = 0:0.01:xmax+0.025;
17.        ya_range = 0:0.01:1;
18.        % 生成随机 x 和 y 坐标
19.        xa = rand(1, 1000*xmax) * (max(xa_range) - min(xa_range))
+ min(xa_range);
20.        ya = rand(1, 1000*xmax) * (max(ya_range) - min(ya_range))
+ min(ya_range);
21.        %生成乙军的散点
22.        xb_range = xmax-0.025:0.01:1;
23.        yb_range = 0:0.01:1;

```

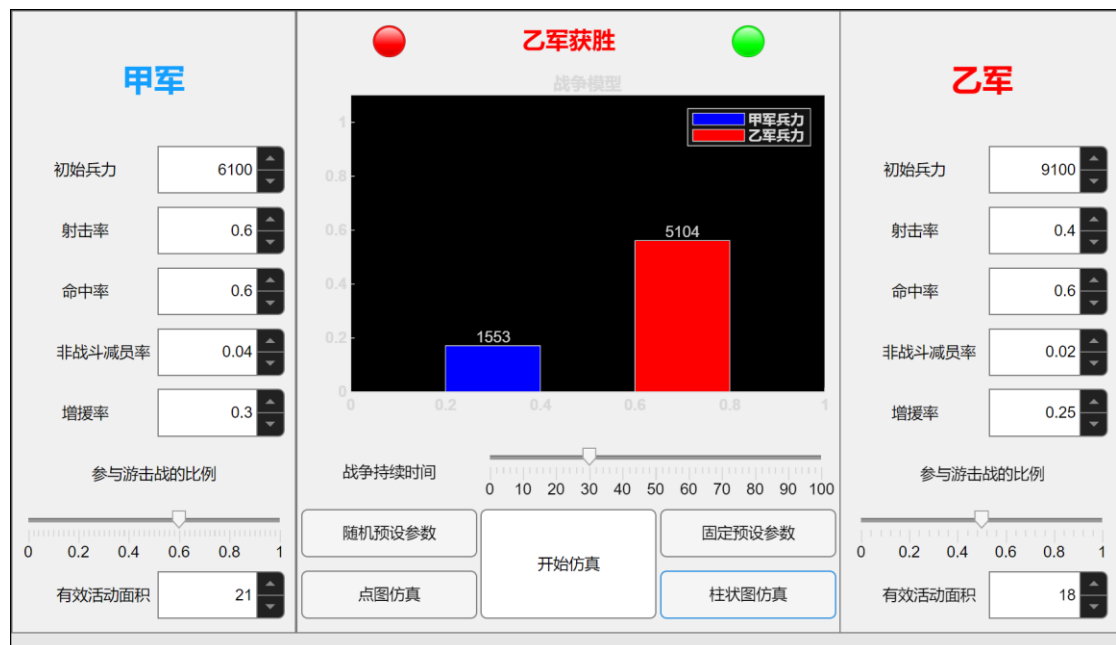
```

24.          % 生成随机 x 和 y 坐标
25.          xb = rand(1, 1000-
1000*xmax) * (max(xb_range) - min(xb_range)) + min(xb_range);
26.          yb = rand(1, 1000-
1000*xmax) * (max(yb_range) - min(yb_range)) + min(yb_range);
27.          %绘制散点
28.          point1=scatter(app.UIAxes, xb, yb, 18, 'r', 'o'); % 'b' 表
示红色, 'filled' 表示填充点
29.          hold(app.UIAxes, 'on');
30.          point2=scatter(app.UIAxes, xa, ya, 18, [0.118,0.565,1], 'o
'); % 'b' 表示蓝色, 'filled' 表示填充点
31.          hold(app.UIAxes, 'on');
32.          legend(app.UIAxes, [point2, point1], {'甲军兵力', '乙军兵力
'}, 'Location','northeast');
33.          %延时
34.          pause(0.05);
35.      end
36.  end

```

4.3.6. 人数变化柱状图的绘制

柱状图将 Y 轴最大值定为两军初始人数的最大值。柱状图的动效绘制思路与散点图相似，在每一帧中计算此时的人数，然后根据比例使用 bar 函数在 app.UIAxes 上绘制当前的图像。最后，在图像的顶端标注当前的剩余人数并延时达到动画效果。



代码如下：

```

1.  function draw_bar(app,t,y)
2.      % 添加坐标轴标签
3.      xlabel(app.UIAxes, ' ');
4.      ylabel(app.UIAxes, ' ');
5.      axis(app.UIAxes, [0,1,0,1.1]);
6.      %设置显示的帧数
7.      K=50;
8.      num0=max(y(1,1),y(1,2));
9.      for i = 1:K
10.         %清空并设置坐标轴范围
11.         cla(app.UIAxes);
12.         %计算此时甲乙两军的数量
13.         Anum=y(i*round((101/K)),1);
14.         Bnum=y(i*round((101/K)),2);
15.         barA=round(Anum/num0,3);
16.         barB=round(Bnum/num0,3);
17.         %绘制散点
18.         bar1=bar(app.UIAxes,0.3,barA ,0.2, 'b'); % 'b' 表示蓝色
19.         hold (app.UIAxes,'on'); % 保持当前图形，以便在同一坐标轴上绘制
第二个柱图
20.         bar2=bar(app.UIAxes,0.7, barB,0.2, 'r'); % 'r' 表示红色
21.         %绘制顶端文字(剩余兵力)
22.         text(app.UIAxes,0.3, barA, num2str(round(Anum)), ...
23.             'HorizontalAlignment', 'center', ...
24.             'VerticalAlignment', 'bottom');
25.         text(app.UIAxes,0.7, barB, num2str(round(Bnum)), ...
26.             'HorizontalAlignment', 'center', ...
27.             'VerticalAlignment', 'bottom');
28.         legend(app.UIAxes, [bar1, bar2], {'甲军兵力', '乙军兵力'}, 'Location', 'northeast');
29.         %延时
30.         pause(0.05);
31.     end
32. end

```


5. 程序运行指南



双击 war app 文件夹下的 war_modol.mlapp 即可运行程序。

使用方法以及可视化动效请观看“演示视频.mp4”。

点击固定预设参数可以直接设置内部固定参数，点击随机预设参数可以生成随机参数

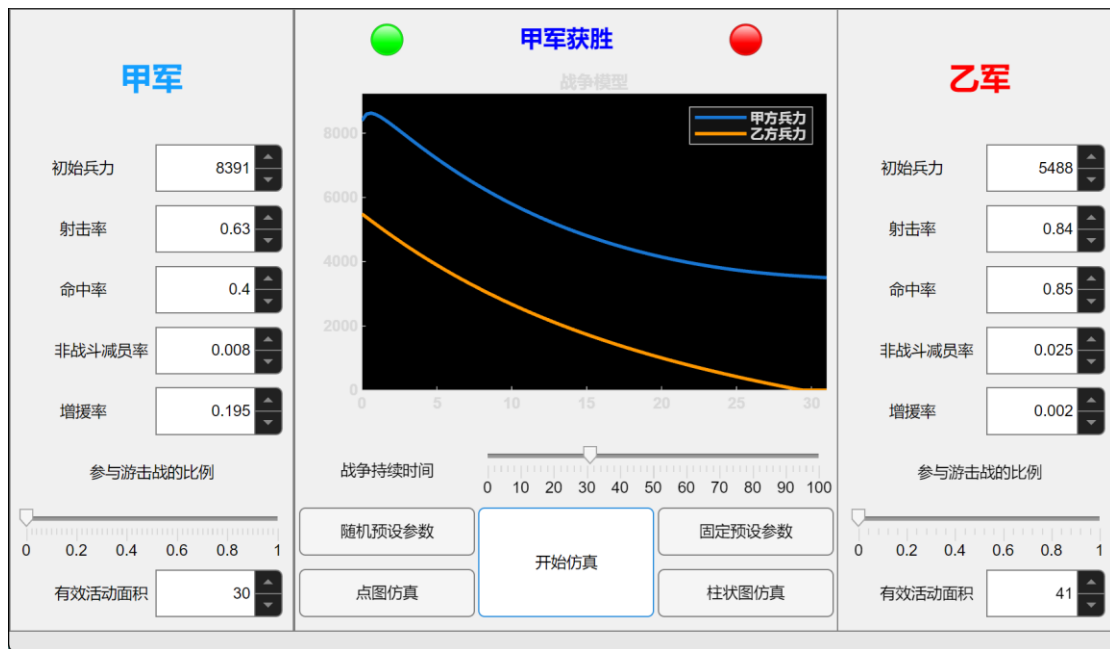
在屏幕上点击输入框或者点击输入框右侧调整箭头可以自定义调整参数

点击开始仿真进行仿真计算并绘制人数-时间曲线图

仿真后点击人数散点图可以观看人数散点变化图，点击人数柱状图可以观看人数变化柱状图。

6. 程序运行实例分析

6.1. 正规战对正规战

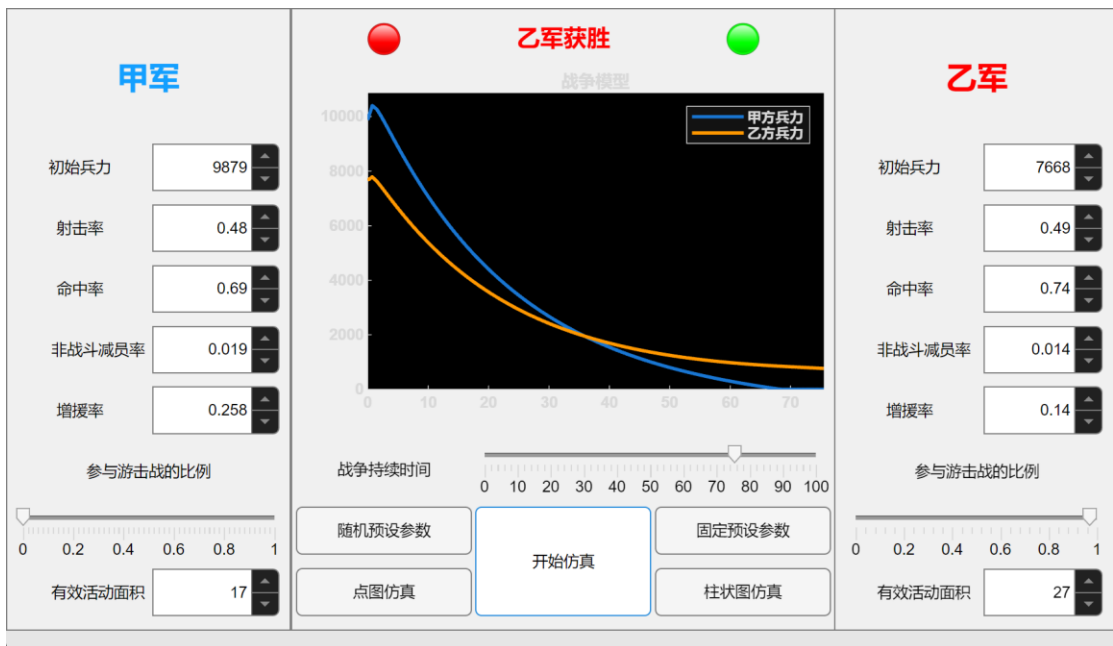


采用一次随机预测出的正规战对战场景，如图，双方参与游击战的比例均为 0，即双方都以正规战作为作战方式。

对比两侧双方的参数可知，在本次战斗中，甲军的初始兵力较多，虽然射击率和命中率较差，但是非战斗减员率较低，增援率也较高，最终凭借人员的压倒性优势战胜了乙方。尽管如此，乙方的射击率和命中率都较高，虽然失败，但也击败了甲军约半数的人数。

因此，本场模拟说明了人员优势和火力优势在战争中都是不可缺少的，与真实情况相符。

6.2. 正规战对游击战



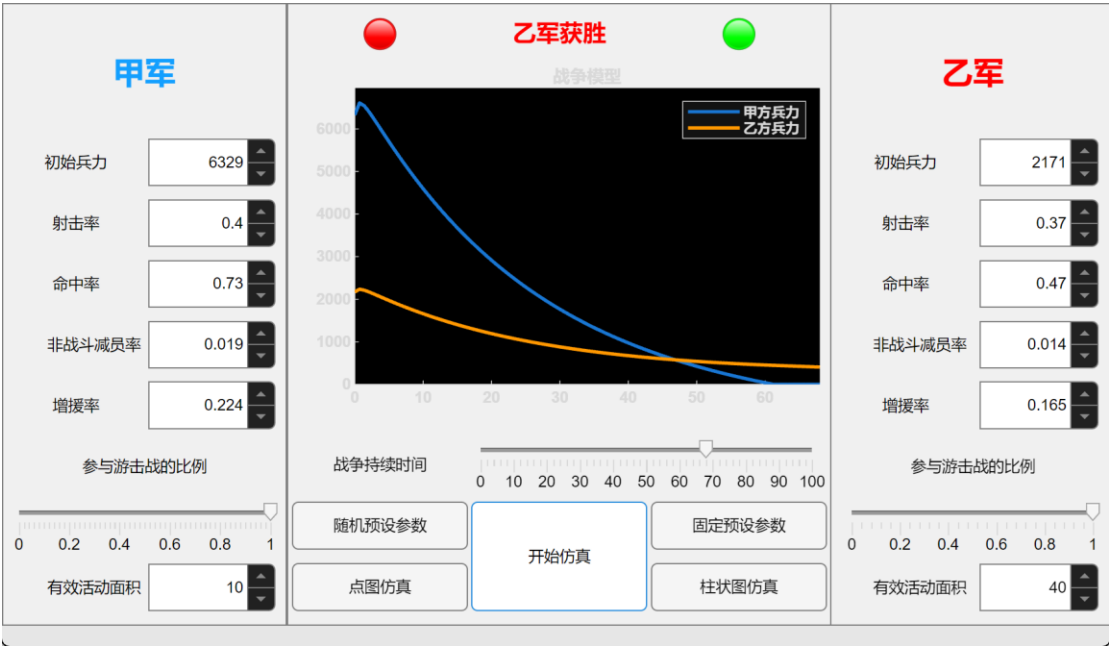
采用一次随机预测出的正规战对战游击战场景，如图，甲方参与游击战的比例为 0，而

乙方的比例为 1，甲方全部以正规军作战，乙方全部以游击队作战。

对比两侧双方的参数可知，在本次战斗中，甲军的初始兵力较多，双方的射击率和命中率相差并不大，且甲军在增援上还略占优势，但是乙军全部采用游击战作战方式，乙军的活动面积远大于甲军，最终，乙军在开始人数相差较多的情况下全部歼灭甲军。

本场模拟战争体现了在游击战作战方式时有效活动面积的重要性，乙军有效活动面积较大，作战灵活，最终战胜了初始实力更强的甲军。符合实际情况。

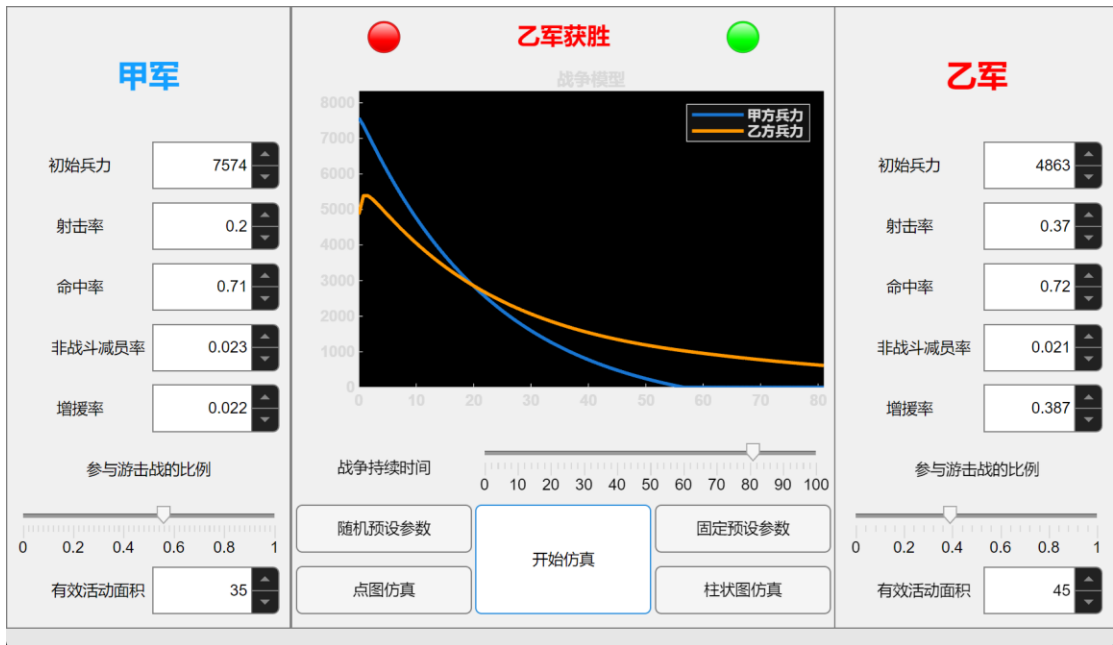
6.3. 游击战对游击战



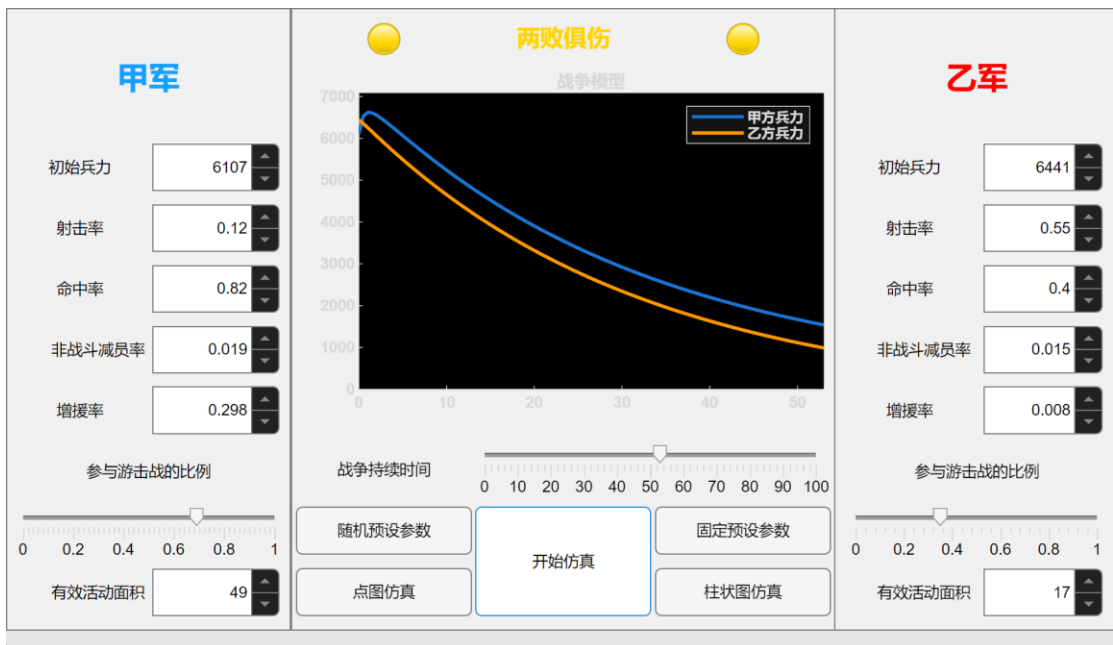
采用一次随机预测出的正规战对战游击战场景，如图，双方参与游击战的比例均为 1，双方全部以游击队作战。

本次战斗中，初始的甲军力量三倍于乙军，呈现人员的压倒性优势，且在命中率上也远超乙军，但是由于乙军的活动面积更大，最终在游击的过程中消灭了甲军。再一次体现了活动面积对于游击战的重要性。

6.4. 混合战



图中双方均以混合战的方式作战，甲军的初始力量略高于乙军，但是乙军的命中率以及活动面积都超过甲方，而甲方在活动面积较为吃亏的游击战中投入了较多的兵力，因此失去了最初的优势，导致了战争的失利。



在另一次的仿真中，甲乙初始军力相持不下，甲军的有效活动面积较大但是射击率太低，乙军的射击率较高但是有效活动面积很小，在游击战中很吃亏，最终，两方力量相近，达成平局的结果。

7. 总结与感悟

本次课设的过程是我第一次完成一套规模较大的 MATLAB 工程的经历。之前虽然也有接触，但是对 MATLAB 的认识也仅仅在于从命令框输入一些简单的计算命令或者编写较小的 M 文件执行计算任务。通过课程的学习和本次的项目实践，我才真正的熟悉了使用 MATLAB 进行数值计算和图形绘制的操作。

在整个课设中，最难最核心的就是建模的部分，战争模型这道题目的模型较为复杂，一开始也对很多输入量的数量级并不了解，运行出来的曲线图非常诡异。我一步一步排查问题，首先发现有时曲线起飞是由于每天增援人数过多导致，于是优化了增援率的计算公式，将 t 和 y 加入到了增援率的考虑中，最终修复了这个错误。之后，我又发现军队的死亡率过高导致所有曲线的变化都很快，于是考虑到了实际中的 t 与模型中的 t 的量级并不一致，因此为符合实际，在死亡率前增设了一个常数 k ，使得模型的预测与实际更贴合。

通过这次的课设，我对 MATLAB 的工程架构、图形绘制都有了初步的了解。也经历了很多发现问题-分析问题-解决问题的过程。感谢王老师一学期的教导，也感谢在我遇到困难问题时帮助我解答疑问的学长。